

# IB Computer Science IA Briefing

## 1. What the IA actually is

"IA" stands for **Internal Assessment**. In IB Computer Science, it is a single, substantial project called the **Computational Solution**. Adhvik must:

1. Identify a **real problem**,
2. **Design and build a working software solution** for it,
3. **Test and evaluate** that solution, and
4. **Document the whole process** in a written report plus a short demonstration video.

It is marked first by the school and then moderated by the IB. It is worth roughly **20% of the final Higher Level grade**. It is not an exam — it is coursework completed over many months, and it is the one part of the course in which the student chooses the topic and demonstrates independent problem-solving.

In plain terms: Adhvik is acting like a junior software developer hired to solve someone's problem, and the report is his record of how he did it.

## 2. Important: Adhvik is on the *new* syllabus

The IB released a **new Computer Science guide** (first teaching September 2025, **first assessment May 2027**). Because Adhvik submits in May 2027, he is on this new version. Most guides, templates, and YouTube videos online still describe the *old* syllabus. Two differences are worth knowing:

- **A real client is no longer compulsory.** Under the new rules the student may choose any problem that interests them; they do not strictly need an external person. *However*, having a real client (which Adhvik does—see Section 6) is still excellent practice and strengthens the project, so he should keep his.
- **The project time allocation increased to about 35 hours**, and there is now a **required short "ethical review"** as part of the evaluation (e.g. data privacy of the people whose information the software stores).

Everything in this briefing reflects the new (2027) requirements. The single most reliable source is the **official IB Computer Science guide (first assessment 2027)**, which Adhvik's school/teacher will have. Use that to confirm exact mark allocations and word counts, as third-party summaries sometimes still mix the old and new versions.

## 3. The five assessment criteria (the heart of the IA)

The report is judged against **five criteria, A to E**, totalling 34 marks. Each criterion corresponds to a stage of the project. Think of them as the chapters of his report.

| Criterion | Stage | What Adhvik must show |
|-----------|-------|-----------------------|
|-----------|-------|-----------------------|

|                                     |        |                                                                                                                                                                                                                 |
|-------------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A — Problem specification</b>    | Define | A clear description of the problem, why it's worth solving, and a list of <b>measurable success criteria</b> the finished software must meet.                                                                   |
| <b>B — Planning</b>                 | Plan   | A record of how he planned the work over time, his research into existing solutions, and justification of his chosen tools/approach.                                                                            |
| <b>C — System overview (design)</b> | Design | Design documents: diagrams (e.g. how data is structured, how screens connect), and a <b>test plan</b> that links each test back to a success criterion.                                                         |
| <b>D — Development</b>              | Build  | The actual coding work: the techniques used, why they were chosen, and evidence of <b>complexity and ingenuity</b> (not just copy-paste code). Includes a <b>short video</b> demonstrating the working product. |
| <b>E — Evaluation</b>               | Judge  | An honest assessment of the finished product against each success criterion, the <b>ethical review</b> , and realistic recommendations for future improvement.                                                  |

**The golden thread:** the success criteria he writes in **A** must reappear in his design (**C**), be visible in the build and video (**D**), and be judged honestly at the end (**E**). An examiner should be able to trace a single requirement from the first page to the last. This "traceability" is what separates a top project from an average one.

#### **Practical requirements to keep in mind**

- **Written report:** concise — roughly 2,000 words across the criteria (the guide gives a suggested word count per criterion; A is about 300 words). It is *not* a long essay; evidence (diagrams, screenshots, short code snippets) does most of the talking.
- **Video:** a screen-recorded demonstration, typically **2–7 minutes**, showing the product actually working. Required at HL.
- **Programming language:** he may use **Python or Java**, among others. Python is the gentler choice for a beginner.
- **Source code** is submitted alongside the report.

#### **4. The process, step by step**

This is the order in which Adhvik should work. Each step feeds the next, so skipping ahead causes problems later.

1. **Identify the problem and confirm it with the client.** Talk to the person who has the problem, take notes, and write down exactly what is broken about their current way of doing things. (*Feeds Criterion A.*)

2. **Write measurable success criteria.** Turn the problem into a checklist of things the software must do — each one specific enough to test later (e.g. "unpaid clients are automatically highlighted on the dashboard"). (*Criterion A.*)
3. **Research and plan.** Look at how others solve similar problems, choose the tools, and lay out a realistic schedule. (*Criterion B.*)
4. **Design the system.** Draw the structure before coding: what data is stored, how it relates, what each screen looks like, and how he will test each feature. (*Criterion C.*)
5. **Build it, in small pieces.** Develop one feature at a time, keeping a running development log and screenshots. (*Criterion D.*)
6. **Test against the success criteria.** Run the tests in the plan, record the results (including any failures), and fix issues. (*Criteria C & D.*)
7. **Get client feedback.** Show the working product to the client and capture their reaction. (*Feeds E.*)
8. **Record the video.** A clean, scripted demonstration of each feature working. (*Criterion D.*)
9. **Write the evaluation.** Honestly judge the product against each success criterion, complete the ethical review, and suggest improvements. (*Criterion E.*)
10. **Assemble, self-check against the criteria, and submit.**

A sensible 12-month rhythm for a May 2027 submission: spend the first couple of months learning to program and locking down the problem and design, the middle stretch building and testing, and the final stretch documenting, filming, and polishing. The biggest risk for students is leaving the build too late, so steady weekly progress matters more than intensity.

## 5. Your role as mentor (and the one rule to protect)

The most useful things a mentor does are: **keep him to a schedule, ask hard questions, review his drafts, and act as a sounding board.** Reviewing his writing for clarity, pointing out a vague success criterion, or asking "how do you know that feature actually works?" are all enormously valuable.

The one rule to protect is **academic honesty**: under IB regulations, the IA must be **Adhvik's own work**. You can guide, ask questions, and give feedback, but you should not write his report sections or code for him. It is completely fine — expected, even — for you to read drafts and give feedback, and for him to then revise them himself. If you are ever unsure where the line is, the school's IB Coordinator can clarify; teachers may give one round of written feedback on a full draft.

You do **not** need a computer science background to mentor him well. Your value is in structure, accountability, and asking the questions a non-expert user of the software would ask.

## 6. Adhvik's client and problem

Adhvik has already chosen his client and problem (from his earlier planning work):

**Client: Sri Hari**, a personal trainer who works across **multiple gym locations** in the city.

**The problem:** Sri Hari currently runs his business out of WhatsApp threads and memory. Three connected things keep going wrong:

1. **Session history** — he struggles to recall which body part a client trained last, because notes are scattered across many chats.
2. **Payments** — monthly payment records are inconsistent, leading to missed collections and disputes.
3. **Scheduling** — when a client cancels at short notice or needs a make-up ("compensation") session, he has no organised way to rearrange his day across different gyms, causing wasted time or double-bookings.

These three problems feed each other: a cancelled session changes the schedule, which changes who was trained, which changes the payment record. So a single connected application is more useful than three separate notes.

**The proposed solution:** a small **web application** (a beginner-friendly approach is Python with the *Flask* framework and a *SQLite* database) that stores client profiles, session history, payment status, and a daily schedule in one place — usable on a phone or in a laptop browser.

**Example success criteria** (the testable checklist that drives everything else):

- A trainer can add a client and view that client's full session history (date + body part trained).
- Monthly payment status is recorded for each client, and **clients with unpaid balances are automatically flagged** on the dashboard.
- A **daily schedule view** lists the day's sessions **grouped by gym location**.
- A cancelled session is **removed from the active schedule but kept in history** as "cancelled".
- A **compensation session** can be created and **linked to the original cancelled one**.
- The app works in a normal browser on **both desktop and mobile** without breaking.

**Where Adhvik is right now:** he is a beginner who needs to learn programming from the start, and he has not yet formally interviewed Sri Hari. His two most urgent next steps are therefore: **(1) sit down and properly interview Sri Hari** (take notes/recording — the IA needs evidence that the problem is real), and **(2) begin learning Python basics** so he can start building.

## 7. The sample dataset (accompanying files)

Alongside this briefing, there is a small, realistic **sample dataset** for the gym-training scenario. It is not Sri Hari's real data — it is invented test data so Adhvik can build and demonstrate the software before (and without exposing) any real client information. He can load it into his app during development and use it to ensure the demonstration video runs smoothly.

It contains four parts that fit together:

- **clients** — the roster of people Sri Hari trains.

- **sessions** — the log of training sessions (date, time, gym, body part, and whether each was completed, scheduled, cancelled, or a compensation session).
- **payments** — each client's monthly payment status (paid / unpaid).
- a ready-to-use **database setup file** that creates the tables and loads all of the above in one step.